language-coverage

Language Coverage

Preston-Check scans source code by `grep`-ing for language-specific patterns. Coverage varies significantly across languages because the catalog grew organically from a Java/TypeScript-heavy fintech codebase. This document audits current coverage, identifies gaps, and lays out the roadmap.

Current coverage matrix

The numbers below count distinct check scripts that include patterns for each language (via `grep --include="*.<ext>"`). A single check can target multiple languages.

LANGUAGE	EXTENSION	CHECKS TARGETING IT	COVERAGE
Java	`*.java`	195	Excellent
TypeScript	`*.ts`, `*.tsx`	108	Excellent
SQL	`*.sql`	36	Strong
Python	`*.py`	11	Limited
Go	`*.go`	11	Limited
JavaScript	`*.js`, `*.jsx`	5	Sparse
Rust	`*.rs`	3	Sparse
Dart	`*.dart`	2	Mobile-only
Terraform	`*.tf`	6	Infra-only
YAML	`*.yml` / `*.yaml`	73	Config-wide
Properties	`*.properties`	4	JVM config
Kotlin	`*.kt`	0	None

LANGUAGE	EXTENSION	CHECKS TARGETING IT	COVERAGE
Ruby	`*.rb`	0	None
C#/.NET	`*.cs`	0	None
PHP	`*.php`	0	None
Solidity	`*.sol`	0	None

Language profiles in `lang/detect.sh` exist for `java`, `go`, `python`, `typescript` / `javascript`, with a generic fallback. There is no Rust profile, which means the auto-detected pattern variables for Rust codebases default to the fallback — most checks then look for Java patterns and find nothing.

Gap analysis

The user's stated requirement is coverage for Node.js, Java, Go, and Rust. Java is solid. Node.js TypeScript is solid; pure JavaScript is thin but the patterns largely overlap with TypeScript. **Go and Rust are real gaps** that need investment before launch claims of "polyglot fintech security scanner" hold up.

Beyond the user's stated four, three more languages deserve coverage:

Kotlin is increasingly common in JVM-based fintechs (notably across Singapore, Korea, and the European neobank stack). Many Kotlin checks can re-use Java patterns by widening file extensions to `*.java *.kt`.

Ruby matters because Stripe, Square Cash, and several major US fintechs are Ruby-on-Rails-heavy. Adding `*.rb` to existing patterns where applicable is low-effort.

Solidity opens the crypto/DeFi vertical. This is a separate scanning domain (smart-contract specific vulnerabilities — reentrancy, integer overflow, oracle manipulation, MEV exposure) and warrants a dedicated sub-suite rather than retrofitting existing checks.

Roadmap

v1.3 — Go and Rust polyglot parity (DELIVERED)

Polyglot parity for Go and Rust shipped in v1.3.0 (P-490..P-505).

Go suite (P-490..P-495): P-490 ignored error returns, P-491 float64 for money (anti-pattern detection plus shopspring/decimal / math/big.Rat recognition), P-492 race conditions on shared state with sync.Mutex / atomic awareness, P-493 constant-time comparison via crypto/subtle, P-494 context cancellation on HTTP and DB calls, P-495 SQL injection via fmt.Sprintf string interpolation.

Rust suite (P-500..P-505): P-500 unwrap()/expect() in production code, P-501 integer overflow without checked_* / saturating_*, P-502 unsafe blocks without SAFETY justification comments, P-503 weak crypto crates (md5, sha-1, deprecated rust-crypto), P-504 unverified serde deserialization, P-505 insecure rand::thread_rng for cryptographic purposes.

These complement Solidity detection added in v1.1.0 and the Solidity- focused crypto suite (P-301..P-310). Combined with `lang/detect.sh` language profiles for Java, TypeScript, JavaScript, Python, Go, Rust, and Solidity, Preston-Check now provides first-class polyglot coverage for the most universal vulnerability classes across the seven languages.

v1.2 — JVM neighbors (Q4 2026)

Kotlin patterns added to Java-targeting checks. Most checks can simply add `*.kt` to their `--include` lists; checks that look for Java-specific annotations (e.g., `@Secured`) need Kotlin equivalents (`@PreAuthorize`, `@Authenticated`).

Scala coverage as a fast-follower — the JVM stack is similar enough that most checks port over with minor pattern adjustments.

v1.3 — Ruby and PHP (Q1 2027)

Ruby on Rails patterns: strong-parameter bypass, `eval` of user input, ActiveRecord SQL injection, ActionController without authentication filters, secrets in `Rails.application.credentials.yml.enc`.

PHP for legacy fintechs: `unserialize()` of user input, weak password hashing (`md5($password)`), magic quotes assumptions, missing CSRF tokens in form handlers.

v1.4 — DeFi and smart contracts (Q2 2027)

A dedicated `checks/defi/` namespace with Solidity-specific checks: reentrancy patterns, integer overflow in pre-0.8 contracts, `tx.origin` authorization, unbounded loops (DoS via gas exhaustion), unchecked external calls, oracle manipulation patterns, missing access controls on `selfdestruct`, and slippage protection on swaps.

This is a meaningful expansion in scope and audience. Worth treating as its own product line within Preston-Check rather than a routine extension.

Contributing language support

The community model is well-suited to growing language coverage. A contributor who writes Rust day-to-day at a fintech can author a single high-quality Rust check, get it through review, and immediately benefit every other Rust fintech in the ecosystem. The `tools/lint-check.sh` linter validates that contributions follow the metadata schema; the maintainer review focuses on correctness and false-positive rates.

To contribute a check for a new language: copy `templates/check.sh`, declare the appropriate `languages` field in the metadata block, and target the language-specific file extension in your `grep --include` pattern. If your language warrants a full profile, also add a `lang/profiles/<language>.sh` file that mirrors the structure of the existing Java and Python profiles.

How language detection affects what runs

The runner uses the dominant-language detection from `lang/detect.sh` to load one language profile via `load_language_profile`. The profile sets pattern variables that some checks consult through `$AUTH_ANNOTATION`, `$BIG_DECIMAL_TYPE`, etc. Checks that hardcode their own patterns ignore the profile entirely.

This means even if your codebase is dominantly Rust, a Java-pattern check runs anyway — it just finds nothing matching, reports `PASS`, and moves on. False negatives are the common failure mode: a check claims `PASS` because it found no Java code, when in fact the equivalent vulnerability exists in your Rust code with different syntax. The roadmap addresses this by adding language-specific sister-checks rather than relying on the profile-variable abstraction alone.